JOHANNES KEPLER
UNIVERSITY LINZ

Author
Daniel Wimmer
12205835

Submission
Institute of Symbolic Artificial Intelligence

Thesis Supervisor
Dr. **Adrian Rebola-Pardo**

February 2026

Intermediate Representation for Modeling Soft and Hard Time Constraints

Bachelor's Thesis

to confer the academic degree of

Bachelor of Science

in the Bachelor's Program

Computer Science

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Abstract

Personal scheduling systems require informal and natural language constraints, such as preferred times and dependencies, to be formalized in order to conform to a common scheme without ambiguities. Current approaches are either difficult to convert into solvable constraints or require the user to plan manually.
Therefore, we define an intermediate language to bridge the gap between complex high-level and solver-ready low-level constraint languages. If existing languages can define a compilation step into the proposed intermediate language, they can be readily translated into constraints that are then supplied to a constraint solver.
The proposed design is independent of any specific solver technology and avoids prescribing how constraints should be solved. Instead, the thesis focuses on defining the structure and semantics of the intermediate language, along with *formal semantics* for validating and comparing solutions, without constraining the methods used to obtain them.

Contents

List of Acronyms

IR Intermediate Representation

JSON JavaScript Object Notation

ITC International Timetabling Competition

NLP Natural Language Processing

LLM Large Language Model

XML eXtensible Markup Language

SMT Satisfiability Modulo Theories

SAT Boolean Satisfiability Problem

MaxSAT Maximum Satisfiability Problem

MaxSMT Maximum Satisfiability Modulo Theories

ILP Integer Linear Programming

CNF Conjunctive Normal Form

BBQ Barbecue

Chapter 1

Introduction

Currently, most scheduling applications, such as Microsoft Outlook or Google Calendar, only supply the options for defining *tasks* with known start times and lengths. These restrictions might not be sufficient for a range of practitioners, as users often need to ensure a task is executed — possibly within a certain time window — without committing to an exact *time slot* upfront. This is especially true for people with time management issues. These individuals struggle to effectively sequence and schedule their tasks, leading to difficulties in determining when and how each task should be completed [17]. Additionally, since personal scheduling is a lot more prone to change in comparison to business schedules, the importance of flexibility is emphasized. Hence, we want to define a scheduling application, which also allows for soft time constraints. This particular use case is not supported by existing tools, even though the necessary technologies already exist.
Tasks with soft time constraints are tasks that

- have time spans with different levels of preference,
- may be optional,
- may establish a specific order in which they must be scheduled,
- and may define specific locations at which they must be performed.

This application should be able to take natural language inputs via the *frontend* and create tasks from that descriptive inputs. These tasks will be represented using the Intermediate Representation (IR), before being given to a constraint solver to construct a viable schedule. This schedule will then be displayed back to the user via the *frontend*. A basic overview of this application and the corresponding information flow is given in Figure 1.1.

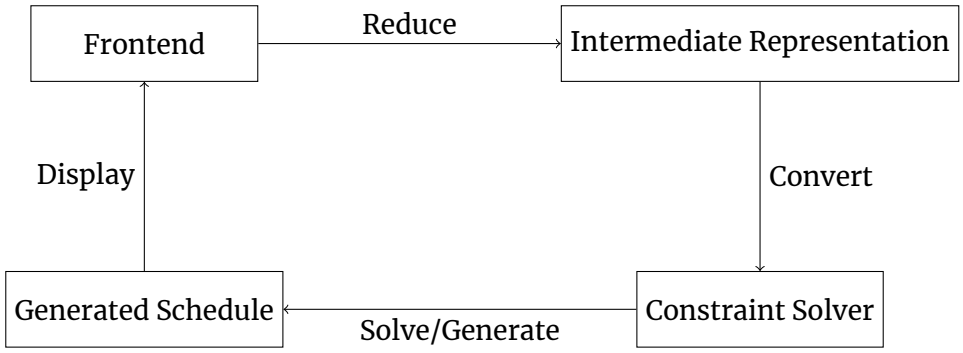

Figure 1.1: Application Overview

The focus of this thesis is on the design and formalization of the IR. Such a representation is beneficial as it provides a clear and unambiguous description of tasks and their associated constraints. This clarity is essential for effective communication between users and the scheduling system, ensuring that the system accurately interprets user requirements. Furthermore, a well-defined IR facilitates the development of algorithms and tools for scheduling, as it provides a structured framework for representing and manipulating scheduling data, instead of having to deal with multiple formats, which are of high complexity. Section 2.3 discusses existing scheduling languages and standards, highlighting the need for one specialized on the personal scheduling domain.

Scheduling is known to be a hard problem, especially constraint scheduling as for example in assigning restricted resources (e.g. rooms) to given professors or courses at a university [9]. The approach presented in this thesis enables the definition of tasks in a minimal form, whilst maintaining expressivity to allow for efficient solving in future work.
We do not define or implement a solver, as the focus of this thesis is on the language design and its formalization. Instead, we provide the necessary framework to bridge the gap between potential ambiguities in wording and solvable constraints.

Specifically, this work tries to answer the following research questions:

- finding a language that provides sufficient expressivity while remaining as simple as feasible,
- giving a notion of solution for a given problem (input language),
- defining how to compare given solutions,
- and showing the relation between the constraints of the language and constraints of solvers.

We evaluated several approaches to identify a suitable solution for the defined research questions. While alternative formulations of the constraints are possible, this work motivates the selected approach and outlines its advantages over other considered options. Additionally, ample examples are provided to illustrate the expressiveness of the proposed representation.

The remainder of the thesis is structured as follows: Chapter 2 introduces the necessary definitions required to understand the subsequent chapters. Chapter 3 presents the language design, including the key design choices and the formal definition of the language. In Chapter 4, the constraints are defined mathematically to characterize what constitutes a valid solution. Finally, Chapter 5 summarizes the findings and presents the conclusions drawn from this work.

Chapter 2

Preliminaries and Related Work

In this chapter we will explain basic concepts such as the representation in time (see Section 2.1) in order to understand what scheduling is. Besides, we will describe what time representation (see Section 2.1) is, and compare hard and soft time constraints in Section 2.2.1, to have a proper basis for the following chapters. Additionally, fundamental terminology is defined to ensure a consistent understanding throughout the thesis, and the rationale behind the selected approaches is outlined, together with a discussion of the limitations of existing solutions in this context.

2.1 Time representations

Representing time in a computer is challenging, because one has to handle dates, times, time zones, and the seasonal clock change, while maintaining a simple representation. To handle instances of this kind, there are two common approaches: continuous and discretized time representations [11]. On the one hand, while continuous time representations allow in general for a more fine-grained representation of time, they might need more complex structures or encodings to be efficiently processed. On the other hand, discretized time representations splits time into fixed size slots (e.g., 30 minutes), which might render very large combinatorial problems. Hence, neither of the two approaches crystallizes the right solution for representing time.
For the particular presentation of time in this thesis, we use discretized time representation — also referred to as *time slots*. This means, we split time into fixed size chunks (i.e. slots). Each *task* can then be scheduled into one or multiple consecutive *time slots*, which will be explained in more detail in Section 3.1.

2.2 Constrained optimization

In general terms, constrained optimization refers to the process of minimizing or maximizing an objective function while adhering to a set of constraints or limitations [3]. In the particular case of scheduling, constraints specify the conditions and limitations that must be satisfied when scheduling tasks [9]. They define what schedules are feasible by restricting the order, timing, and allocation of tasks and resources. Typical constraints arise from resource capacities, task dependencies, deadlines, and mutual exclusivity requirements, ensuring that tasks do not conflict and that system limitations are respected. By

formalizing these restrictions, constraints reduce the solution space to schedules that are both valid and practically implementable. Therefore, we can describe the general problem as

$$
\begin{aligned}
\text{minimize} \quad & f(x) \\
\text{subject to} \quad & g_i(x) \leqslant 0, \quad i = 1, \dots, m \\
\text{and} \quad & h_j(x) = 0, \quad j = 1, \dots, p \\
\text{where} \quad & f : \mathbb{R}^n \to \mathbb{R} \text{ is the objective function} \\
& g_i : \mathbb{R}^n \to \mathbb{R} \text{ are the inequality constraints} \\
& h_j : \mathbb{R}^n \to \mathbb{R} \text{ are the equality constraints}
\end{aligned}
$$

In this formulation, $f(x)$ represents the objective function to be minimized, while $g_i(x)$ and $h_j(x)$ denote the inequality and equality constraints, respectively. The goal is to find the optimal values of x that minimize the objective function while satisfying all specified constraints.

2.2.1 Hard versus Soft Time Constraints

In constrained optimization, constraint functions restrict the solution space and must not be violated. When considering scheduling constraints such as deadlines and dependencies between tasks, we distinguish between hard and soft time constraints. While both types restrict the scheduling of tasks, they differ in their level of strictness and flexibility. Hard time constraints are non-negotiable restrictions that a solution must satisfy in order to be valid; these constraints are therefore modeled using the $h_j(x)$ or $g_i(x)$ functions in the constrained optimization formulation. Hard time constraints are "hard" in two senses: they are temporally strict — i.e., tasks must be scheduled at a specific time and for a specific duration — and they are mandatory, meaning that violating them renders a solution invalid for the given problem [9].

In contrast, soft time constraints describe tasks that are desirable but not strictly required to be scheduled. Such tasks may be scheduled within a broader time range and, potentially, with flexible durations (the latter not being part of this thesis). An example of a soft time constraint is a preference such as "I would like to read a book every Friday, if possible." Solutions that include these tasks should be preferred over those in which they are omitted. Consequently, soft constraints are modeled within the objective function $f(x)$, which is to be minimized, by introducing a notion of cost — as described in Section 2.2.2 — to quantify the desirability of scheduling such tasks.

2.2.2 Scheduling Assignment and Cost

A scheduling assignment is defined as a mapping of *tasks* to specific *time slots*. Each *task* has a fixed *duration*, which determines the number of consecutive *time slots* required to schedule it. Throughout this thesis, a scheduling assignment is also referred to as a solution. Each solution is associated with a cost, which is defined as a metric that quantifies how preferable a given assignment is compared to other assignments for the same problem (the same set of tasks) [14]. Lower cost values indicate better solutions. Ideally, a solution with

a cost of zero represents the best achievable outcome. Consequently, a solution with a cost of 15 is considered superior to a solution with a cost of 200. A precise formal definition of the cost function, along with further explanations, is provided in Chapter 4.

2.2.3 Scheduling problem

Scheduling refers to the task of assigning a finite set of time-constrained events to limited resources under a set of restrictions [2]. In the context of this thesis, it denotes the assignment of tasks — characterized by fixed durations, relative temporal dependencies, and possibly other constraints — to specific days and times (i.e. time slots), which constitute the limited resources. Further, a valid schedule most adhere to the previously mentioned constraints, while optimizing the cost of an assignment (i.e. mapping of tasks to time slots) for a given problem (see Section 2.2.2). Formally, scheduling problems can therefore be viewed as constrained optimization problems, where the assignment of tasks to time slots constitutes the decision variables, hard scheduling requirements are modeled as constraints, and preferences or soft requirements are captured in the objective function.

2.2.4 Solving Methods for Constrained Optimization

In the following chapter we will introduce three prominent solving paradigms for constrained optimization problems. Namely, Maximum Satisfiability Problem (MaxSAT) (as an extension of SAT), Maximum Satisfiability Modulo Theories (MaxSMT) (as an extension of SMT) and Integer Linear Programming (ILP) as the three most prominent paradigms. Their differences in expressiveness, solver technology, and application domains justify their inclusion in a brief overview of automated reasoning approaches. The three following chapters will introduce the most basic concept of the respective solver paradigms. Since the field of possible constraint solvers is vast, we restrict ourselves to these three options to ensure the focus of the thesis stays on the core concept of our Intermediate Representation.

Maximum Satisfiability Problem

MaxSAT enhances SAT to be able to handle constrained optimization problems [1]. SAT handles propositional logic formulas, which are representations of Boolean variables possibly combined with logical operators like **AND** ($\wedge$), **OR** ($\vee$), and **NOT** ($\neg$). Most solvers are specialized on formulas in Conjunctive Normal Form (CNF), which means there is a conjunction $\left(\bigwedge_{j=1}^{M} c_j\right)$ of clauses, which in turn are disjunctions $\left(\bigvee_{i=1}^{N} p_i\right)$ of literals (Boolean variables or their negations). An example formula in CNF would be: $a \wedge (b \vee \neg c)$ with a, b, c being the Boolean literals and a and $(b \vee \neg c)$ being the clauses. This conjunctive normal form can be obtained from any proportional logic formula in polynomial time via the Tseitin transformation [18].
In contrast to basic SAT, MaxSAT has the addition of soft clauses. We define

a set of m weighted clauses $\phi = \{(C_1, w_1), \dots, (C_m, w_m)\}$, which provides a notion of significance to each clause [1]. Specifically, the higher the weight w_i of a clause C_i, the more important it is to satisfy the clause. If one wants to define hard clause, one has to simply set $w = \infty$. The goal of MaxSAT is to find an assignment for ϕ with as little cost as possible. The cost is defined as the sum of weights of all falsified clauses in the MaxSAT formula. This is a good fit for our problem definition, as we want an assignment with as little cost as possible, while satisfying all hard constraints.

Maximum Satisfiability Modulo Theories

To extend the expressive power of SMT to optimization problems, we consider MaxSMT. SMT augments SAT by interpreting propositional variables over a background theory (i.e. over the Integers) [15, 16]. In particular, SMT mends the gap between expressiveness and efficiency of automatically solving constraint problems. By introducing the likes of equality, arithmetics and uninterpreted functions we get a more powerful language, which makes expressing problems easier [7, 16]. On the other hand, this expressiveness comes with a trade-off in solver efficiency and hence, we need to introduce specialized theory solvers. Similarly to SAT, SMT most commonly uses the CNF. Which in terms of SMT is a conjunction of clauses, which in turn are literals or theory atoms (i.e. linear arithmetic) [10]. An example formula in SMT would be: $\Phi = x + y \leqslant 10 \wedge (z = x * 2 \vee \neg(y < 5)) \wedge f(x) = 4$. In SMT we see the usage of arithmetics, which is not possible in SAT, which makes SMT a more expressive language. Such a formula Φ is satisfiable if a suitable interpretation for x, y, z, f can be found that meet the constraints of the formula. MaxSMT extends this idea by the addition of soft constraint — similarly as in MaxSAT [10]. Hence, we define a set of weighted formulas $\phi = \{(F_1, w_1), \dots, (F_m, w_m)\}$, where each formula F_i has a weight w_i. The goal of MaxSMT is identical to the goal of MaxSAT, which is to find the assignment for ϕ with as little cost as possible, while maintaining all hard constraints, which also fits the problem definition.

Integer Linear Programming

ILP is designed to either minimize or maximize an objective function, while holding inequality and equality constraints or keeping integrality restrictions on variables [5]. An example of an ILP problem would be: $z = \min(c^T x | Ax \leqslant b, x \in \mathbb{Z}^n)$. Here, $c^T x$ is the objective function to minimize and z represents the corresponding minimum value. $Ax \leqslant b$ are the constraints, and $x \in \mathbb{Z}^n$ enforces integrality on the variables. As we can see, ILP is specialized on linear relations, which makes it a good fit for our problem domain.
In summary, while all three paradigms vary in expressiveness and solver technology, they are capable of expressing our problem domain. Additionally, combining multiple paradigms can be more efficient than relying on isolated solvers [19]. Hence, specific choice for the most appropriate solving method, or combination of methods, demands further investigation and experimentation.

2.3 Existing Scheduling Languages and Standards

After introducing the basic ideas underlying the IR, it is necessary to clarify which formats and standards already exist and why our language still addresses issues that remain unsolved by current approaches. One of the most widely used standards for scheduling is the iCalendar format, defined in RFC 5545 [8]. It specifies a set of rules for representing text-based, potentially recurring events and tasks, and is widely adopted in both professional and personal scheduling applications. Owing to its extensive rule set, iCalendar is considerably more expressive than the language (or set of constraints) proposed in this thesis. This expressiveness, however, comes at the cost of making the format difficult to translate into a solver-compatible representation. Additionally, RFC 5545 is not able to handle complex constraints of tasks, which do not have a static set date and need to be scheduled dynamically, which is a core requirement of the application proposed in Chapter 1.

Another relevant format is defined by the International Timetabling Competition (ITC) [12], whose primary purpose is to specify timetabling problems involving the assignment of courses to dedicated rooms at specific times. Similar to our approach, it relies on discretized time slots. Nevertheless, this format is overly complex for our use case. In particular, the ITC format is designed to handle multiple constrained resources simultaneously (e.g., rooms and instructors), whereas our problem involves only a single resource — the individual allocating the tasks. Consequently, our model requires fewer degrees of freedom than the ITC format.

Other existing scheduling languages are designed for professional areas such as school time tabling [6] and sports time tabling [14]. However, the goal of the approach described in this thesis is to be applicable to everyday life (i.e. personal schedules).

2.4 Data Interchange Formats for Task Representation

As one needs a way to define and store tasks, we need a language to represent said tasks. For ease of use we decided against defining our own language from scratch. Instead, we use JavaScript Object Notation (JSON) to represent our basic language. The decision is justified by the widespread use of JSON [4]. If one wanted to instead use eXtensible Markup Language (XML), it can be converted to JSON and back [13]. As the sole requirement of the task representation is to store information in an adequate way, it can easily be mapped to other file formats, as long as it can describe the structure and fields of the tasks without ambiguities.

Chapter 3

Methodology and Language Design

The aim of this chapter is to describe the design of the proposed language without focusing on semantics. We provide the required constraints alongside detailed design choices for each field contained in our language.
As we are tackling a personal scheduler, we need to cover the most common requirements one might need to define in everyday scenarios. Therefore, we identified the following basic requirements:

- *Duration* defines how long a task will take
- *Assignments* defines at which point in time the task may be scheduled
- *Dependencies*: depict that one task has to be scheduled after another
- *Locations*: emphasize efficiency, as unnecessary traveling can be avoided

The above constraints were found to be the most important and therefore are covered in the IR. These constraints are explained in this chapter.

3.1 Core Concept of the Intermediate Representation

Horizon

The `horizon` field is used to specify the length of time (i.e. the number of time slots) the solver can define `tasks` to. Without defining such a stopper, one would have to take infinite time into account which is unnecessary for a scheduling application. Most of the time appointments tend to be stacked up towards the near future and become less frequent the further one looks into the future, as their existence is not known yet. This goes both ways, as a personal scheduler should also tend to schedule the bulk of tasks in the near future in order to be able to compensate for tasks, which occur later and shorter before their deadlines. Therefore, this method does not noticeably restrict the possible solutions as most of the `tasks` are bundled towards the near future. To schedule further into the future, the maximum time slot may be increased, which may lead to longer runtimes.

Tasks

The *tasks* keyword describes the core object of personal scheduling. Each task represents an activity that may or needs to be scheduled at a specific time. To express constraints between tasks, each task must be uniquely identifiable such that other constraints (e.g. dependencies) can refer to them unambiguously.

Duration

The *duration* keyword describes the length of a specific task. This is a fundamental aspect of scheduling, as it determines how much time a task will occupy in the schedule.

Importance

Importance in general describes how critical it is to schedule a given task. The higher the importance, the more cost is incurred for not scheduling it. This allows the solver to prioritize certain tasks over others, ensuring that the most critical tasks are scheduled, if possible.

Urgency

As stated in Section 2.1, we aim to bulk `tasks` in the near future, leaving the schedule largely empty for the distant future. This is desirable because most of the `tasks` are not known beforehand. Hence, we also need to ensure that our schedule handles as many of the `tasks` as soon as possible to help with the fact that at a later time, one needs space for the `tasks` one does not know yet. Therefore, *urgency* is introduced to encourage the solver to schedule tasks as early as possible. The later a task is scheduled the higher the urgency cost incurred for it.

Time Slots

One *time slot* is a discretized unit of time, for example 15 minutes or 10 seconds. Starting with zero the *time slots* can describe an exact point in time if the exact point in time is known for any of the *time slots*. It is used to abstract away the concept of time. When chaining together multiple time slots without gaps, a specific point in time can be represented.

Example 3.1. *Let A be an arbitrary Task. A should be scheduled on December the 31st at 6 PM and takes one hour. If we compile this exactly on December 31st at 6 PM and one time slot is discretized to 15 minutes, we can conclude that this task should be scheduled at time slot 4, 5, 6, and 7, as shown in Table 3.1.*

index	0	1	2	3	4	5	6	7	...
start	17:00	17:15	17:30	17:45	18:00	18:15	18:30	18:45	...
end	17:15	17:30	17:45	18:00	18:15	18:30	18:45	19:00	...

Table 3.1: Ranges of time slots

When using this approach, we handle only the time of interest (i.e. the future), while ensuring the concept of time is discretized and abstracted away as early as possible in the conversion process from natural language to constraints.
The `timeSlots` field defines when a task can be scheduled, identifying preferred windows and periods to avoid.

Dependencies

The `dependencies` field models relationships between `tasks`. When specifying schedules in practice, we repeatedly encounter two kinds of situations:

1. A task `B` is optional, but it only makes sense to schedule it if another task `A` has also been scheduled (e.g., "buy groceries" only matters if "barbecue" happens).
2. A task `B` must be scheduled after a task `A` has been completed, potentially within a preferred (or required) time frame (e.g., "clean up" after "barbecue", ideally not days later).

We capture both situations uniformly by viewing a *dependency* as a cost that is evaluated when the dependent task `B` is scheduled. The cost depends on (i) whether the prerequisite task `A` is scheduled at all, and (ii) if both are scheduled, on the time gap between finishing `A` and starting `B`. If `B` is scheduled but `A` is not, the dependency incurs an `unmetCost` of the dependency defined by `B` (potentially infinite). Otherwise, the `intervals` in the dependency describe which time gaps are admissible or preferred and what cost they induce. This high-level interpretation is made precise later in Section 4.1 and Section 4.3.

Location

We introduce the `location` field to allow practitioners to schedule certain tasks together. This is especially useful for tasks tied to a physical location, but it can also group activities that can be done in parallel, such as listening to the news while coding.

3.2 Data Conversion and Preprocessing

Dealing with the complex concept of time is a major challenge in scheduling. Hence, we handle things like time zones, daylight saving time, and the conversion of dates to time slots in the front-end (i.e. assume the input is already converted to time slots). This allows for easier conversion and lighter representation in the IR.

Additionally, as the IR is not designed to handle ambiguity, which is common in natural language, we require the front-end to resolve any ambiguities before converting the user-input to the IR. This includes, for example, determining the exact cost for scheduling task A at time slot X instead of time slot Y, or deciding on the importance of a task. Furthermore, *tasks*, *dependencies*, and *locations* have to be uniquely defined per scheduling problem, and hence need a unique ID, which also needs to be generated in the front-end.

3.3 JSON Language Specification

The fields defined in Section 3.1 form the core of the proposed language, resulting in the following JSON structure:

⟨*file*⟩ a JSON object containing exactly the following fields:

- "horizon" a JSON integer H defining the number of available time slots (indexed from 0 to $H - 1$)
- "tasks" a JSON object of type dictionary with a JSON string as a key (a unique task ID) and a JSON object of type ⟨*task*⟩ as a value

⟨*task*⟩ a JSON object containing exactly the following fields:

- "duration": a JSON integer denoting the number of contiguous time slots to allocate
- "importance": a JSON integer or the JSON string "Infinity" denoting the cost incurred if the task is left unscheduled
- "urgency": a JSON integer encoding preference for early scheduling; later scheduled tasks incur higher urgency cost
- "timeSlots": a JSON array of type ⟨*interval*⟩ specifying admissible (and optionally preferred) scheduling windows via interval–cost pairs
- "dependencies": a JSON array of type ⟨*dependency*⟩ (optional)
- "location": a JSON object of type ⟨*location*⟩ (optional)

⟨*dependency*⟩ a JSON object with exactly the following fields:

- "task": a JSON string (the ID of the prerequisite task)
- "intervals": a JSON array of type ⟨*interval*⟩ specifying preferred time frames via interval–cost pairs
- "unmetCost": a JSON integer or the JSON string "Infinity" denoting the cost incurred if the dependency is violated

⟨*location*⟩ a JSON object with exactly the following fields:

- "id": a JSON integer (location identifier)
- "unmetCost": a JSON integer or the JSON string "Infinity" denoting the cost incurred if the desired grouping by location is not achieved

⟨*interval*⟩ a JSON object with exactly the following fields:

- "interval": a JSON array of exactly two JSON integers $[a, b]$ denoting the half-open interval $[a, b)$ over time-slot indices, where $0 \leqslant a < b \leqslant H$
- "cost": a JSON integer representing the cost incurred if the task is scheduled within this interval
- "unmetCost": a JSON integer or the JSON string "Infinity" (where applicable, the cost incurred if the associated constraint cannot be satisfied)

In order to be able to properly handle ⟨*interval*⟩, we considered three approaches for efficiently representing this data:

1. Array of bits where each position represents a specific time slot; the bit flag indicates whether the task is permitted in that slot.
2. Array of intervals where each entry represents a contiguous range of time slots available for task allocation.

3. Array of pairs, each consisting of an interval and an associated cost, allowing for prioritized scheduling based on preference or resource constraints.

Bit arrays are easy to handle and efficient to store, but their size grows linearly with the `horizon`. Since admissible time slots are typically grouped into larger windows, a more compact representation is an array of intervals, which can also easily be converted to a bit array[1]. We use half-open intervals such as $[0, 10)$ (i.e. "interval" in the above specification), meaning 0 is included while 10 is excluded. For expressiveness, we additionally associate a cost with each interval (i.e. "cost" in the above specification) and use an array of interval–cost pairs.

3.3.1 Schema Verification

A JSON schema is used to make the language explicit and verifiable. This schema defines without ambiguity what fields are required, what types are valid for these fields and other constraints such as minimum and maximum length of arrays. An excerpt of the schema file showing the definition of `unmetCost` from `dependencies` is shown below[2]:

```
...
"unmetCost": {
    "oneOf": [
        { "type": "integer" },
        {
            "type": "string",
            "enum": [ "Infinity" ]
        }
    ]
},
...
```

In addition, a schema validation tool is needed for testing. We decided to use C# with the *Newtonsoft.Json.Schema* library[3] for our implementation to verify the input files.

3.4 Expressiveness and Limitations

To emphasize the expressiveness of our language, given the little number of constraints, we provide examples to illustrate how the defined fields can be used to express complex scheduling scenarios. For instance, variable length tasks can be represented by splitting a task into multiple subtasks and using

[1] Half-open interval $[x, y) = [b_0, b_1, \dots]$ where $b_i = \begin{cases} 0, & 0 \leqslant i < x \\ 1 & x \leqslant i < y \\ 0, & \text{else} \end{cases}$.

[2] The complete schema file can be found in our repository: https://github.com/h0p3zZ/BScThesis/blob/dev/_prog/JsonVerifier/schema.json

[3] Newtonsoft.Json.Schema library: https://www.newtonsoft.com/jsonschema

dependencies and importance to enforce the minimum duration while preferring the longer durations.

Example 3.2. *To model a gym session that requires a minimum of 45 minutes but ideally lasts one hour, we can define two subtasks with durations of 45 minutes and 15 minutes, respectively. The first task is enforced using a high importance value (or ∞ if mandatory). The second task is constrained to follow immediately after the first using dependencies; a high dependency cost (or ∞ if mandatory) applies if there is a gap between them. However, a lower cost is assigned if the second task is omitted entirely, offering flexibility in the schedule while still preferring the full hour when possible.*

Similarly, the defined fields allow for the representation of reoccurring tasks. This can be achieved by defining multiple tasks for the same activity, each with its own time slot constraints to ensure they are scheduled at different times.

Example 3.3. *Consider a scenario where one wants to go to the gym every Monday. This can be achieved by defining multiple tasks, each representing a gym session for a specific Monday. By signing appropriate time slot constraints to each task, we can ensure that they are scheduled on different Mondays, thus effectively modeling the recurring nature of the activity.*

Furthermore, the representation can describe a problem where task B should be scheduled after task A, but task A must not be scheduled. This can be achieved by setting the importance of task A to a low value, while setting the dependency cost of task B on task A to ∞. This way, the solver is encouraged to schedule task A and then task B — if possible, while still being forced to schedule task B if task A is not scheduled.

Example 3.4. *Imagine a barbecue scenario in which you have two tasks: "'Buy Groceries" and "Barbecue". By setting the dependency cost of "Barbecue" on "Buy Groceries" to ∞, we can ensure that they are scheduled together. Furthermore, by assigning a low importance value to "Buy Groceries" and a high importance value to "Barbecue", we can encourage the solver to schedule "Buy Groceries" if possible, while still allowing for the possibility of scheduling "Barbecue" without "Buy Groceries" if necessary.*

However, this comes with the caveat that the representation cannot express certain constraints. For example, assigning two dependent tasks in a both or nothing manner is not possible (i.e. either schedule both with a low cost or not schedule both with a medium cost).

Example 3.5. *A scenario like baking an oven split up into two steps. The first step is to prepare the dough, which takes 30 minutes. The second step is to bake the dough, which takes 45 minutes. Both steps are dependent on each other, meaning that if one of them is not scheduled, the other one should also not be scheduled. This cannot be expressed in our representation, as we cannot enforce that both tasks are either scheduled together or not at all.*

Additionally, the representation does not cover traveling costs apart from the primitive location cost. This means that the cost of moving from one location

to another is only incurred if the first task is allocated immediately before the second task. Hence, two tasks with a big distance between them can be scheduled with a low cost, as long as they are scheduled more than zero time slots apart. This limitation was deliberately accepted, as a rigorous solution to the problem of modeling traveling costs would require a deep dive into the theory of scheduling with sequence-dependent setup times.

Example 3.6. *Consider two tasks, "Go to the Gym" and "Go to the Supermarket", which are located at two geographically distant locations. If now, the assignment would schedule them with a gap of one, the cost of moving between these two locations would not be considered and therefore, the solution would be suboptimal.*

Chapter 4

Formal Framework

For the definitions introduced in Chapter 3 to be well-defined, we need to establish formal mathematical semantics. The proposed semantics are specified in three levels of abstraction:

1. Data definition (see Section 4.1), which specifies the data required to uniquely define a problem instance.
2. Search space (see Section 4.2), which describes the space of all feasible solution configurations.
3. Semantics (see Section 4.3), which formally specifies what constitutes a valid solution for a problem defined in Section 4.1. In addition, this level defines comparison of solutions in the form of cost described in Section 4.3.1.

For global use and notational simplicity, we introduce the following definitions:

- $\mathbb{N}_u = \mathbb{N}_0 \cup \{u\}$, where u denotes an undefined or unscheduled value.
- $\mathbb{Z}_u = \mathbb{Z} \cup \{u\}$, where u denotes an undefined or unscheduled value.
- $\mathbb{N}_\infty = \mathbb{N}_0 \cup \{\infty\}$, where ∞ denotes infinite cost.

4.1 Data Definition

It is necessary to specify the information required by the solver to enable the solution of a given problem. Therefore, we define the following functions both in natural language and mathematical terms.

- `tasks`: T is a finite set of tasks.
  The set naturally translates to the dictionary of tasks contained in the JSON representation. Each $t \in T$ represents a unique task. For example, "SetupLocation" corresponds to a task defined in Example 4.1.
- `horizon`: $H \in \mathbb{N}_0$
  This is a direct mapping to the `horizon` key in the JSON. In the example defined in Example 4.1 the `horizon` is set to 50.
- `duration`: $\text{dur}: T \longrightarrow \mathbb{N}_0$
  Is the duration of a task in time slots, directly mapping to the `duration` key in the JSON. In our example, we have $\text{dur}(\text{SetupLocation}) = 2$.

- `time slot function`: $\text{slot}\colon \mathsf{T} \times \mathbb{N}_u \longrightarrow \mathbb{N}_\infty$
  which maps every task at a given time slot to a cost. This cost includes the time slot cost at the given time, the importance cost (if not scheduled), and the urgency given the corresponding assignment. The function is a combination of multiple fields of the IR. If task A is not scheduled ($\text{slot}(A, u)$) the `importance` cost is incurred. Otherwise, it checks all defined time slots of the task and if the given assignment overlaps with at least one of the defined intervals, it returns the cost of all overlapping intervals accumulated plus an `urgency` cost calculated as $(x + 1) \cdot \texttt{urgency}$. To compute the accumulated interval cost for a scheduled task A at start time slot $x \in \mathbb{N}_0$, we consider the time window occupied by the task, $[x, x + \text{dur}(A))$. Let $\mathcal{I}(A)$ be the set of intervals specified for A in `timeSlots`. Each interval $I \in \mathcal{I}(A)$ contributes proportionally to how much of the task window overlaps with I. Concretely, with overlap length $|I \cap [x, x + \text{dur}(A))|$, we define the accumulated interval cost as

  $$\sum_{I \in \mathcal{I}(A)} \texttt{cost}(I) \cdot \frac{|I \cap [x, x + \text{dur}(A))|}{\text{dur}(A)}.$$

  Intuitively, if half of a task's duration lies inside a particular interval, then that interval contributes half of its `cost`. Considering the example specified in Example 4.1, we construct $\text{slot}(\text{BuyGroceries}, 29) = \texttt{interval.cost} + \texttt{urgency} = 0 + (29 + 1) \cdot 1 = 30$ as an example, which provides a cost of 30 for this exact input. The example assignment used in the example is the same as in Section 4.2.

- `dependency function`: $\text{dep}\colon \mathsf{T} \times \mathsf{T} \longrightarrow \mathbb{Z}_u \longrightarrow \mathbb{N}_\infty$
  It returns the cost of a task's dependencies, given another task — if a dependency exists between them. This cost can also be infinite, if specified as such in the `unmetCost`. If the second task is not scheduled, the dependency function does not incur any cost, as the dependency itself is not violated. However, if the first task is not scheduled, but the second task is, the `unmetCost` is added to the total cost. This function maps every pair of tasks, where the second one has to be scheduled to another function, which in turn maps the relative distance of their assignments to a cost. The function checks all defined intervals for the dependency and if the relative distance of the assignments is within one of the defined intervals, it returns the cost of that interval. If multiple intervals are found, the costs of all applicable intervals are accumulated. If no valid interval is found it again returns the `unmetCost`. Also, if the tasks do not share a dependency, the resulting cost function is just a constant function to zero. In this case, when looking at the Barbecue (BBQ) example, we have for example $\text{dep}(\text{BuyGroceries}, \text{BBQ})(35 - (29 + 3)) = \text{dep}(\text{BuyGroceries}, \text{BBQ})(3) = 0$.

- `locations`: L is a finite set.
  The set contains all unique location IDs defined for all tasks. In Example 4.1, we have $\mathsf{L} = \{1, 2\}$, assuming "SetupLocation" has location ID 1 and "BBQ" has location ID 2.

- `location function`: $\text{loc}\colon \mathsf{T} \longrightarrow \mathsf{L} \times \mathbb{N}_\infty$
  The function specifies whether a task has a set location and what the cost would be of going to this specific location. Specifically, the cost is inferred, if the location of the task is different from the location of the

previous task and both tasks are scheduled immediately after one another. For convenience, we define $\text{loc}(t) = (\text{loc}_L(t), \text{loc}_c(t))$. The set of locations is defined by all unique location IDs for all tasks. Additionally, the `location function` maps every task to a tuple of both the location ID and the cost of going there. If a task has no defined location, the function returns $(\text{null}, 0)$, meaning there is no *location* and therefore no cost.

These definitions provide a comprehensive framework for representing the necessary data to define a scheduling problem unambiguously. The above functions and sets form the basis of the problem, in order to be able to define valid solutions and semantics in the subsequent sections. The specific representation defined in Chapter 3 can be directly mapped to these formal definitions, as definition above is strictly more expressive than the keywords defined in the IR. Thus, every valid IR specification can be translated into a well-defined problem instance using the above definitions.

Example 4.1 (Exemplary Scheduling Scenario). *For our personal scheduling application, we aim to support the specification of high-level constraints as comprehensively as possible. To illustrate the expressive power of our language and to allow for a more concrete understanding, we present the following example. Consider the task of planning a BBQ with friends. This requires scheduling several interdependent tasks, such as purchasing groceries, setting up the `location`, preparing the barbecue itself, and cleaning up afterward. In addition, certain temporal constraints must be respected. For example, the groceries should not be purchased too early before the barbecue, as the food may otherwise spoil. Using our language, these high-level constraints can be expressed as follows[1]:*

```
{
    "horizon": 50,
    "tasks": {
        "BuyGroceries": {
            "duration": 3,
            "importance": 100,
            "urgency": 1,
            "timeSlots": [
                { "interval": [0, 50], "cost": 0}
            ],
            "location": {
                "id": 2,
                "unmetCost": 50
            }
        },
        "SetupLocation": {
            "duration": 2,
            "importance": 100,
            "urgency": 1,
            "timeSlots": [
                { "interval": [0, 50], "cost": 0}
            ],
            "location": {
```

[1]The BBQ example can be found in our repository: https://github.com/h0p3zZ/BScThesis/blob/dev/_prog/SolutionVerifier/thesis_problem.json

```
                "id": 1,
                "unmetCost": 10
            }
        },
        "BBQ": {
            "duration": 10,
            "importance": "Infinity",
            "urgency": 0,
            "timeSlots": [
                { "interval": [35, 45], "cost": 0}
            ],
            "dependencies": [
                {
                    "task": "SetupLocation",
                    "intervals": [
                        { "interval": [0, 6], "cost": 0},
                        { "interval": [6, 10], "cost": 15}
                    ],
                    "unmetCost": 500
                },
                {
                    "task": "BuyGroceries",
                    "intervals": [
                        { "interval": [0, 6], "cost": 0},
                        { "interval": [6, 20], "cost": 100}
                    ],
                    "unmetCost": 500
                }
            ],
            "location": {
                "id": 1,
                "unmetCost": 10
            }
        }
    }
}
```

In this example, three `tasks` are defined. Intuitively, the "main" task is the BBQ itself, which has to be scheduled exactly at time slot 35 to 45 (due to the `duration` of 10 and it being the only valid interval) and `importance` is infinity. The task "SetupLocation" has to be scheduled at time slot 0 to 9, because valid intervals are [0,10), when combining the two intervals. Furthermore, the task is scheduled with cost zero when scheduled before time slot 6, otherwise a cost of 15 is incurred. Similarly, "BuyGroceries" has to be scheduled at time slot 0 to 19, with cost zero when scheduled before time slot 6, otherwise a cost of 100 is incurred. Both `tasks` have to be scheduled before the BBQ, otherwise a high `unmetCost` (500) of the `dependencies` is incurred. Although the `tasks` "SetupLocation" and "BuyGroceries" are not mandatory, but they are important. If it is not possible to schedule these `tasks`, one can still ask someone else attending the barbecue to do it.

4.2 Search Space

Having established the information required to solve the problem, we now characterize the general structure of a solution. A solution is represented by an assignment function that maps each task to either a concrete time slot or an undefined value indicating that the task is not scheduled. Formally, we define an assignment as a function `assignments` $a \colon T \longrightarrow \mathbb{N}_u$ and A is the set of all possible functions $T \longrightarrow \mathbb{N}_u$.

Example 4.2 (Exemplary Assignment). *Combining the problem definition of Example 4.1 with the notion of an assignment, we provide an example of a possible solution configuration. The following assignment specifies that the task "BuyGroceries" is scheduled at time slot 29, "SetupLocation" at time slot 32, and "BBQ" at time slot 35:*

```
{
    "BuyGroceries": 29,
    "SetupLocation": 32,
    "BBQ": 35
}
```

In this representation, each task is associated with a specific time slot, or u if it is not scheduled. In the present example, all tasks are assigned to concrete time slots. Consequently, the corresponding assignment function a satisfies, for example, $a(\textit{BuyGroceries}) = 29$.

4.3 Semantics

We can now bring together the problem definition and the notion of a solution by specifying the conditions under which a candidate solution is considered valid. To this end, we define $T_a = \{t \in T \text{ such that } a(t) \neq u\}$, i.e., the set of tasks that are scheduled in the assignment a.
Additionally, we employ the Kronecker delta function δ to support the computation of location-related costs. This function compares two values and returns 1, if they are the equal, and 0 otherwise.
For notational simplicity, we also define a notion of difference between two values in $\mathbb{N}_u$:

$$- : \mathbb{N}_u \times \mathbb{N}_u \longrightarrow \mathbb{Z}_u$$

$$n_1 - n_2 = \begin{cases} n_1 - n_2 & \text{for } n_1, n_2 \in \mathbb{N}_0 \\ u & \text{else} \end{cases}$$

Conceptually, an assignment $a \in A$ is valid iff:

- For all pairs of distinct assigned tasks t_1, t_2 the durations must not overlap.
  $a(t_1) + \text{dur}(t_1) \leqslant a(t_2) \vee a(t_2) + \text{dur}(t_2) \leqslant a(t_1)$

To illustrate we perform the calculations based on Example 4.2:

$$a(\text{BuyGroceries}) + \text{dur}(\text{BuyGroceries}) = 29 + 3 = 32 \leqslant 32 = a(\text{SetupLocation}) \quad \text{True}$$
$$a(\text{BuyGroceries}) + \text{dur}(\text{BuyGroceries}) = 29 + 3 = 32 \leqslant 35 = a(\text{BBQ}) \quad \text{True}$$
$$a(\text{SetupLocation}) + \text{dur}(\text{SetupLocation}) = 32 + 2 = 34 \leqslant 35 = a(\text{BBQ}) \quad \text{True}$$

- For every assigned task t the assigned time slots must not exceed the horizon.
  $a(t) + \mathit{dur}(t) < H$
  Again, considering assignments based on Example 4.2:

$$a(\text{BuyGroceries}) + \text{dur}(\text{BuyGroceries}) = 29 + 3 = 32 < 50 = H \quad \text{True}$$
$$a(\text{SetupLocation}) + \text{dur}(\text{SetupLocation}) = 32 + 2 = 34 < 50 = H \quad \text{True}$$
$$a(\text{BBQ}) + \text{dur}(\text{BBQ}) = 35 + 10 = 45 < 50 = H \quad \text{True}$$

- For a given assignment a we need the `cost function`, to be finite:
  $c(a) \in \mathbb{N}_0$
  We define the cost function as $c(a) = c_1(a) + c_2(a) + c_3(a)$, where c_1, c_2, and c_3 are the three components of the cost function corresponding to the time slot costs, dependency costs, and location costs, respectively. For the three components of the cost function this translates to:

  1. Every task $t \in T$ must have a finite cost at its assigned time slot:

$$c_1(a) = \sum_{t \in T} \text{slot}(t, a(t)) \tag{4.1}$$
$$c_1(a) \in \mathbb{N}_0$$

     The first component of the cost function (i.e. Equation (4.1)) aggregates the cost incurred by each task at its assigned time slot. If a task is scheduled the urgency is incorporated automatically, for example through the term $(a(t)+1)\cdot\mathit{urgency}$. In contrast, if a task is not scheduled the corresponding importance cost is added. An illustrative example of this behavior was already provided in Section 4.1 in the discussion of the `time slot function`.

  2. For every distinct pair of tasks $t_1 \in T$ and $t_2 \in T_a$ where the second task is scheduled, the dependency cost must be finite:

$$c_2(a) = \sum_{t_1 \in T, t_2 \in T_a} \text{dep}(t_1, t_2)(a(t_2) - (a(t_1) + \text{dur}(t_1))) \tag{4.2}$$
$$c_2(a) \in \mathbb{N}_0$$

     The second part (i.e. Equation (4.2)) sums up the dependency costs for every pair of tasks. The second task has to be assigned, because the task holds the dependency. The first task may be scheduled, though scheduling it is optional. This flexibility is useful for expressing constraints such as the following: “If I have time to buy the food for the BBQ, it should be done at most one day before it”. In this case the task chronologically scheduled sooner can be committed but also enforced,

if needed. The cost is computed based on the relative temporal distance between the assignments of the two tasks. An example illustrating this mechanism was already provided in Section 4.1, in the discussion of the `dependency function`.

3. Before defining the third part of the cost function we introduce the `location change function`, which is one iff:

$$\begin{aligned}\mu(l_1, l_2, n_1, n_2) =& (1 - \delta(l_1, l_2)) && \text{two locations differ}\\ & \cdot \delta(n_1, n_2) && \text{two time slots are the same}\\ & \cdot (1 - \delta(\text{null}, l_1)) \cdot (1 - \delta(\text{null}, l_2)) && \text{neither location is “null”}\end{aligned}$$

Reiterating the above definition, the function μ checks whether two tasks are scheduled immediately after one another (i.e. n_1 and n_2 are the same) and whether they have different locations (i.e. l_1 and l_2 differ). If both conditions are satisfied and neither of the tasks has a “null” location, the function returns one, otherwise it returns zero. This function is used to determine whether a location change occurs between two tasks that are scheduled back-to-back, which in turn incurs a cost if the locations differ.
With this definition we can now define the third and final part of the cost function $c(a)$.
For every distinct pair of scheduled tasks $t_1, t_2 \in T_a$ the location cost must be finite:

$$c_3(a) = \sum_{t_1, t_2 \in T_a} \text{loc}_C(t_2) \cdot \mu(\text{loc}_L(t_1), \text{loc}_L(t_2), a(t_1) + \text{dur}(t_1), a(t_2)) \tag{4.3}$$

$$c_3(a) \in \mathbb{N}_0$$

The third part (i.e. Equation (4.3)) sums up the location costs for each pair of scheduled tasks that follow one another immediately. If both tasks share the same location no cost is added. Otherwise, the cost of driving to the location of the second task is added to the total cost, given that neither location is “null”. Additionally, we provide an example for the location by using “SetupLocation” and “BBQ” from Example 4.2. Assuming both tasks have different locations defined, we calculate:

$$\begin{aligned}& \text{loc}_C(\text{BBQ}) \cdot (1 - \delta(\text{loc}_L(\text{SetupLocation}), \text{loc}_L(\text{BBQ}))\\ & \cdot \delta(a(\text{SetupLocation}) + \text{dur}(\text{SetupLocation}), a(\text{BBQ}))\\ =& 10 \cdot (1 - \delta(1, 1)) \cdot \delta(32 + 2, 35)\\ =& 10 \cdot (1 - 1) \cdot \delta(34, 35)\\ =& 10 \cdot 0 \cdot 0 = 0\end{aligned}$$

4.3.1 Computing Cost

The `cost function` can be split into three distinct parts:

To enable the comparison of different solutions, we introduce a quantitive evaluation criterion in the form of a cost metric. Specifically, we use the functions defined in Section 4.3 (i.e. a_1 from Equation (4.1), a_2 from Equation (4.2)

and a_3 from Equation (4.3)) to define the `cost function` $c\colon A \to \mathbb{N}_\infty$:

$$c(a) = c_1(a) + c_2(a) + c_3(a)$$

Evaluating all of the above parts gives us the total cost of the given assignment (a), given the underlying definition of tasks (the problem).

Similarly to Section 3.3.1, we have implemented the computation of cost and validity checking of a given assignment to a specific problem in C#. The implementation can be found in our repository[2]. Specifically, the cost computation can be found in the *SolutionVerifier* project[3]. Finally, we also provide the assignment example from Section 4.2 in the repository to test the cost computation[4].

[2] Link to our repository: https://github.com/h0p3zZ/BScThesis
[3] *SolutionVerifier* program: https://github.com/h0p3zZ/BScThesis/tree/dev/_prog/SolutionVerifier
[4] Assignment file: https://github.com/h0p3zZ/BScThesis/tree/dev/_prog/SolutionVerifier/thesis_assignment.json

Chapter 5

Conclusion and Outlook

In summary, this thesis introduces the fundamental notions underlying the scheduling problem addressed by the proposed IR in Chapter 1. Next, in Chapter 2 we present the language design, including justified choices made during its development. Furthermore, we introduce basic concepts of constrained optimization and representation of time in general. Chapter 3 demonstrates the design of our language and the reasoning behind certain decisions, highlighting the descriptive power of our language. After defining the language, we formalize the constraints mathematically in Chapter 4, giving a clear definition of what a solution is and how to compare different solutions. In the process of defining the formal framework, we identify required properties, which serve as inputs for the solver. Subsequently, we establish the formal definition of a valid solution and how to compare different solutions using a cost function, which can be used to find optimal solutions.

The main contribution of this thesis is the design and formalization of a highly expressive constraint language for personal scheduling. We have defined a set of constraint primitives that are significantly more expressive than they initially appear. As demonstrated in Chapter 3, these primitives can be combined to express tasks with variable durations, conditional scheduling, and preference-based time windows. Crucially, we have provided rigorous formal semantics for all constraints in Chapter 4, establishing a clear mathematical foundation for what constitutes a valid solution and how to compare different solutions through well-defined cost functions.

The defined IR successfully addresses the goals outlined in Chapter 1 by enabling flexible scheduling across broad time spans while respecting temporal constraints. This language is designed to serve as an intermediate translation step between any high-level language and low-level solver-ready constraints. Both the high- and low-level languages are replaceable and not bound to any specific implementation, allowing for a wide range of applications as long as the high-level language can be compiled into the IR and the low-level solver can handle the defined constraints.

As the proposed language only poses as an intermediate step, and it does neither handle solving the schedule itself nor provide a user interface to convert natural language input into IR. Hence, future work consists of implementing a solver that can handle the defined constraints is of utmost importance. The defined formal framework serves as a basis for such an implementation. An implementation — or rather its output — can be checked with an existing implementation of a solution verifier[1]. Second, benchmarking different

[1]Repository with the implementation of a syntax compiler for the IR and a solution verifier: https://github.com/h0p3zZ/BScThesis/tree/dev/_prog

types of solvers with our defined constraints represents an interesting avenue for future research. Third, implementing a compiler from a high-level language into our IR is beneficial to showcase the usability of our language. Finally, an intuitive way of defining tasks in a high-level language must also be provided. Such an input is most likely expressed in natural language, which can be parsed using Natural Language Processing (NLP) and/or Large Language Model (LLM) techniques.

Bibliography

[1] Carlos Ansótegui and Joel Gabas. 2013. Solving (Weighted) Partial MaxSAT with ILP. In *CPAIOR*. Volume 13, pp. 403–409.

[2] Kenneth R Baker and Dan Trietsch. 2018. *Principles of sequencing and scheduling*. John Wiley & Sons.

[3] Dimitri P Bertsekas. 2014. *Constrained optimization and Lagrange multiplier methods*. Academic press.

[4] Pierre Bourhis, Juan L. Reutter, and Domagoj Vrgoč. 2020. JSON: Data model and query languages. *Information Systems*, 89, 101478. ISSN: 0306-4379. DOI: https://doi.org/10.1016/j.is.2019.101478. https://www.sciencedirect.com/science/article/pii/S0306437919305307.

[5] S. Chaudhuri, R.A. Walker, and J.E. Mitchell. 1994. Analyzing and exploiting the structure of the constraints in the ILP approach to the scheduling problem. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2, 4, 456–471. DOI: 10.1109/92.335014.

[6] Emir Demirović. 2017. *SAT-based approaches for the general high school timetabling problem*. PhD thesis. Technische Universität Wien.

[7] Leonardo de Moura, Bruno Dutertre, and Natarajan Shankar. 2007. A Tutorial on Satisfiability Modulo Theories: (Invited Tutorial). In *International conference on computer aided verification*. Springer, pp. 20–36.

[8] Bernard Desruisseaux. 2009. Internet Calendaring and Scheduling Core Object Specification (iCalendar). RFC 5545. (2009).

[9] A.I. Diveev and O.V. Bobr. 2017. NP-Hard Task Schedules and Methods of Its Decision. In *2017 IEEE 11th International Conference on Application of Information and Communication Technologies (AICT)*, pp. 1–5. DOI: 10.1109/ICAICT.2017.8686990.

[10] Katalin Fazekas, Fahiem Bacchus, and Armin Biere. 2018. Implicit Hitting Set Algorithms for Maximum Satisfiability Modulo Theories. In *Automated Reasoning*. Didier Galmiche, Stephan Schulz, and Roberto Sebastiani, (Eds.) Springer International Publishing, Cham, pp. 134–151. ISBN: 978-3-319-94205-6.

[11] Christodoulos A. Floudas and Xiaoxia Lin. 2004. Continuous-time versus discrete-time approaches for scheduling of chemical processes: a review. *Computers & Chemical Engineering*, 28, 11, 2109–2129. ISSN: 0098-1354. DOI: https://doi.org/10.1016/j.compchemeng.2004.05.002. https://www.sciencedirect.com/science/article/pii/S0098135404001401.

[12] 2019. ITC 2019 Data Format. Accessed: 2025-12-21. https://www.itc2019.org/format.

[13] Robin La Fontaine. 2017. Making a Difference by Processing JSON as XML. In (*Balisage Series on Markup Technologies*, volume 19). Balisage: The Markup Conference 2017 (August 1–4, 2017). Washington, DC. DOI: 10.4242/BalisageVol19.LaFontaine01.

[14] Martin Mariusz Lester. 2022. Pseudo-Boolean optimisation for RobinX sports timetabling. *Journal of Scheduling*, 25, 3, 287–299. ISSN: 1099-1425. DOI: 10.1007/s10951-022-00737-7. https://doi.org/10.1007/s10951-022-00737-7.

[15] Joao Marques-Silva. 2008. Practical applications of boolean satisfiability. In *2008 9th International Workshop on Discrete Event Systems*. IEEE, pp. 74–80.

[16] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. 2006. Solving SAT and SAT Modulo Theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T). *J. ACM*, 53, 6, (November 2006), 937–977. ISSN: 0004-5411. DOI: 10.1145/1217856.1217859. https://doi.org/10.1145/1217856.1217859.

[17] Gil D. Rabinovici, Melissa L. Stephens, and Katherine L. Possin. 2015. Executive Dysfunction. *Continuum (Minneapolis, Minn.)*, 21, 3, 646–659. DOI: 10.1212/01.CON.0000466658.05156.54.

[18] G. S. Tseitin. 1983. On the Complexity of Derivation in Propositional Calculus. In *Automation of Reasoning: 2: Classical Papers on Computational Logic 1967–1970*. Jörg H. Siekmann and Graham Wrightson, (Eds.) Springer Berlin Heidelberg, pp. 466–483. DOI: 10.1007/978-3-642-81955-1_28. https://doi.org/10.1007/978-3-642-81955-1_28.

[19] Jialu Zhang, Chu-Min Li, Sami Cherif, Shuolin Li, and Zhifei Zheng. 2025. Integer Linear Programming Preprocessing for Maximum Satisfiability. *arXiv preprint arXiv:2506.06216*.